***COLLEGE:ABASAHEB GARWARE COLLEGE (AUTONOMOUS)***

**T.Y.BSc.(Computer Science)**

**PROJECT REPORT ON**

# Cybersecurity Malware Project

ACADEMIC YEAR:2025-2026

***SUBMITTED BY***:

SANJANA SUBHASH KAMBLE

# 1. Introduction

Portable Document Format (PDF) files are ubiquitous in digital communication, used across offices, schools, and websites due to their ability to preserve formatting across devices. However, their trusted nature makes them a prime target for cyber-attacks. Attackers often embed malicious JavaScript, hidden objects, or exploit code within these files to harm systems or steal data when opened.

This project addresses these threats by developing an automated toolkit using **Python** to perform **static analysis**. This process allows for the inspection of a file's internal structure—detecting phishing links and embedded scripts—without actually executing the file in an unsafe manner.

## 2. Infrastructure of Project

The project follows a structured workflow to ensure comprehensive analysis without risking system infection.

### Workflow Architecture

1. **Load PDF File:** The user provides the file path, and the tool loads the document into the environment.
2. **Extract Metadata:** The system gathers file information such as Author, Creation Date, and Producer.
3. **Scan for Suspicious Keywords:** It searches for specific keywords often associated with malicious behavior.
4. **Analyze JavaScript:** The tool identifies if any embedded JavaScript is present, which is a common vector for automated attacks.
5. **Extract URLs & IPs:** The system pulls Indicators of Compromise (IOCs), such as web links and IP addresses, from the file.
6. **Risk Scoring:** Based on the elements found, the tool calculates a numerical risk score.
7. **Generate Report:** A final assessment classifies the file as **Low, Medium, or High risk**.

## 3. Comprehensive System Requirements

**Software Stack:**

**Python 3.10+:** Essential for modern library compatibility.

**PyPDF2 Library:** Specifically used for its ability to access metadata and internal object dictionaries without rendering the file.

**Visual Studio Code (VS Code):** Recommended for debugging and script management.

**Environment:** Can run on a standard local machine or within a **Virtual Machine (VM)** like VirtualBox (highly recommended for actual malware analysis to isolate the host)

**Hardware Stack:**

**RAM:** 4GB is sufficient for scanning small files, but 8GB is better for processing large, multi-page documents.

**Storage:** Minimal space is required for the script itself (less than 5MB), but space is needed for the Python environment and libraries.

## 4. Results & Output (What the user sees)

When the analysis is complete, the tool generates a report structured like this:

**File Identity:** Name, size, and MD5 hash (for checking against known malware databases).

**Security Summary:** * "JavaScript detected: Yes (Count: 2)"

"Automatic Actions: Yes"

"External Links Found: 3"

**Risk Level:** A color-coded or text-based warning (e.g., **"WARNING: HIGH RISK"**)

```
Command Prompt

[+] PDF Loaded Successfully

--- PDF Metadata ---
/Author:
/CreationDate: D:20260201001644+05'30'
/ModDate: D:20260201001644+05'30'
/Producer: Microsoft: Print To PDF
/Title: *This is a test PDF file. - Notepad

--- Keyword Scan ---
[+] No suspicious keywords found

--- JavaScript Analysis ---
[+] No JavaScript detected

--- IOC Extraction ---
URLs Found: {'http://example.com'}
IP Addresses Found: None

--- Risk Analysis ---
Risk Score: 2
Threat Level: LOW RISK

--- Malware Analysis Report ---
File Name: sample.pdf
Risk Score: 2
Threat Level: LOW RISK
Recommendation: Do NOT open this PDF if risk is Medium or High.

C:\Users\sanjana\OneDrive\Desktop>
```

## 5.Malware Analyzer Code

```
import PyPDF2

import re

import os


class PDFMalwareAnalyzer:

    def __init__(self, file_path):

        self.file_path = file_path

        self.suspicious_keywords = [
```

```
                        '/JS', '/JavaScript',
'/OpenAction', '/Launch',
                        '/RichMedia', '/XFA',
'/EmbeddedFile'
                    ]
                    self.risk_score = 0
                    self.found_triggers = []


                def analyze(self):
                    if not
os.path.exists(self.file_path):
                        print("Error: File not found.")
                        return


                    print(f"--- Analyzing:
{os.path.basename(self.file_path)} ---")


                    try:
                        with open(self.file_path, 'rb')
as f:
```

```
                reader =
PyPDF2.PdfReader(f)


                # 1. Metadata Extraction

                meta = reader.metadata

                print(f"[+] Producer:
{meta.producer if meta else 'Unknown'}")

                print(f"[+] Pages:
{len(reader.pages)}")


                # 2. Keyword & JavaScript
Scan

                # We read the raw content
to find encoded/hidden triggers

                f.seek(0)

                content =
f.read().decode('ascii', errors='ignore')


                for keyword in
self.suspicious_keywords:

                    count =
content.count(keyword)
```

```
                    if count > 0:

self.found_triggers.append(f"{keyword} ({count} times)")

                        self.risk_score += (count * 2)


                # 3. URL Extraction (IOCs)

                urls = re.findall(r'https?://(?:[-\w.]|(?:%[\da-fA-F]{2}))+', content)

                unique_urls = list(set(urls))



self.display_results(unique_urls)


            except Exception as e:

                print(f"Analysis Error: {e}")


        def display_results(self, urls):

            print("\n--- Findings ---")
```

```
                if self.found_triggers:
                    print(f"Suspicious Elements: {', '.join(self.found_triggers)}")
                else:
                    print("No common suspicious keywords found.")


                if urls:
                    print(f"Extracted URLs: {len(urls)} found")
                    for url in urls[:5]: # Show first 5
                        print(f"  -> {url}")


                # Risk Assessment Logic
                print(f"\nTotal Risk Score: {self.risk_score}")
                if self.risk_score == 0:
                    print("Assessment: LOW RISK (Clean)")
                elif self.risk_score <= 6:
```

```
                    print("Assessment: MEDIUM RISK (Inspect manually)")
                else:
                    print("Assessment: HIGH RISK (Potentially Malicious!)")


        # Usage
        if __name__ == "__main__":
            # Replace 'sample.pdf' with the path to the PDF you want to test
            path = "sample.pdf"
            analyzer = PDFMalwareAnalyzer(path)
            analyzer.analyze()
```

### 3. How the Code Works:

**Binary Reading:** The file is opened in `'rb'` mode because malware often hides in the non-text portions of a PDF

**Keyword Search:** It looks for `/OpenAction` or `/JavaScript`. If a PDF is designed to run code the moment it is opened, these keywords will be present in the file's internal dictionary.

**Regex (Regular Expressions):** The `re` module scans for patterns that look like web addresses. This identifies "Phishing" links that might lead to credential-stealing websites.

**Risk Scoring:** The script assigns points based on how many dangerous elements it finds. A simple document shouldn't have multiple `/JavaScript` or `/Launch` commands.

### 4. Sample Output Description

When you run this code, your console will output a report similar to this:

> **Metadata:** Shows if the file was made by a known tool (like Microsoft Print to PDF).
>
> **Suspicious Elements:** Lists triggers like /JS found in the code.
>
> **URLs:** Lists any web addresses found inside the document structure.
>
> **Final Score:** A numerical value that tells the user whether to trust the file.

## 6.Future Enhancements

While the current version of the project effectively performs **Static Analysis**, the following features could be implemented to make it a professional-grade security tool:

> **Dynamic Analysis (Sandboxing):** Integrating a "Sandbox" environment where the PDF can be opened in a safe, isolated virtual space to observe its behavior in real-time. This would detect "Zero-Day" exploits that static scanning might miss.
>
> **Deep Learning & AI Integration:** Using Machine Learning models (like Random Forest or Neural Networks) trained on thousands of known malicious and benign PDFs. This would allow the system to predict if a file is dangerous based on complex structural patterns rather than just keywords.

**De-obfuscation Engine:** Attackers often "hide" their JavaScript code using hex encoding or compression to bypass simple scanners. A future version could include an automated script to decode and "unmask" this hidden code for deeper inspection.

**Integration with VirusTotal API:** The tool could automatically send the file's hash (MD5/SHA-256) to VirusTotal to see if other antivirus engines worldwide have already flagged it as a threat.

**Support for Other Document Formats:** Expanding the infrastructure to analyze Office documents (Word, Excel) which, like PDFs, often contain malicious Macros.

**Web-Based Dashboard:** Developing a Graphical User Interface (GUI) or a web portal where users can "Drag and Drop" files for instant analysis instead

---

## 7.Conclusion

The **PDF Malware Analysis** project successfully demonstrates the critical role of automated security screening in modern computing. By leveraging **Python** and the **PyPDF2** library, we have developed a functional toolkit capable of identifying hidden threats—such as malicious JavaScript, unauthorized "OpenAction" commands, and suspicious external links—without risking the integrity of the host system.

**Key Achievements of the Project:**

**Safety First:** The project highlights the importance of **Static Analysis**, allowing users to inspect a file's "DNA" without ever executing the potentially harmful code.

**Efficiency:** The automated nature of the script significantly reduces the time required for a security analyst to vet a document manually.

**Risk Quantification:** By implementing a numerical Risk Scoring system, the project provides a clear, actionable assessment (Low, Medium, or High) for non-technical users.

**Educational Value:** Beyond the software, this project provides a deep understanding of how attackers exploit trusted file formats and how those same formats can be defended using logical programming.